

Object Oriented Programming

Why OOP?

IT560



Sourish Dasgupta

Assistant Professor, DA-IICT, Gandhinagar

<https://daiict.ac.in/faculty/sourish-dasgupta>

Why not stick to Functional Programming?

Drawback of Procedural Programming	How FP Addresses It	How OOP Addresses It
Tight coupling between data and functions	• Immutability • Pure functions	• Encapsulates data and behavior within objects
Global state and side effects	• Immutability • Pure functions	• Restricted access via Encapsulation • Does not inherently avoid side effects
Code duplication and lack of reusability	• Reusable pure functions • Higher-order functions	• Inheritance • Polymorphism
Poor modularity and maintainability	• Function composition & modularity	• Self-contained classes and objects
Difficulty in scaling large applications	Scales well (via functional decomposition) for: • Mathematical problems • Stateless systems	• Modularity • Encapsulation • Inheritance

Why not stick to Functional Programming?

Drawback of Procedural Programming	How FP Addresses It	How OOP Addresses It
Difficulty in modeling real-world systems	<i>No good answer!</i>	• Real-world entities as objects and classes
State management challenges	• Immutability (i.e., avoids mutable state) • State mgmt. simplified!	• Objects are encapsulated • Object state managed locally, allowing easy updates
No clear abstraction boundaries	• Higher-order functions	• Encapsulation • Inheritance • Interfaces
Poor team collaboration for large projects	• Modular code • May require expertise to maintain clarity in team settings	• Code as objects and classes • Easy for code delegation

What happens when systems are large?

Scalability in Functional Programming



Scaling large applications: Statelessness (btw, Python begins!)

```
def calculator(operation, a, b):  
    if operation == "add":  
        return a + b  
    elif operation == "subtract":  
        return a - b  
    elif operation == "multiply":  
        return a * b  
    elif operation == "divide":  
        return a / b if b != 0 else "Error: Division by zero"  
    else:  
        return "Error: Unsupported operation"  
  
# Example usage:  
print(calculator("add", 2, 3))          # Output: 5  
print(calculator("multiply", 4, 5))     # Output: 20; Independent of  
previous call
```

- **Independence:** Each interaction is standalone
- **No Memory:** The system does not retain data or "state" between requests
- **Scalability:** Easy to scale because there's no need to manage or synchronize state

Scaling large applications: Statelessness via Function Composition

```
from functools import reduce

def apply_operations(initial_value, operations):
    return reduce(lambda acc, op: op(acc), operations, initial_value)

# Define operations as functions
operations = [
    lambda x: add(x, 5),      # Add 5
    lambda x: multiply(x, 2), # Multiply by 2
    lambda x: subtract(x, 3), # Subtract 3
    lambda x: divide(x, 7),   # Divide by 7
]

result = apply_operations(0, operations) # Start with 0
print(f"Final result: {result}") # Output: 1.0
```

Can any system be modeled as stateless? Ex: Shopping Cart

1. Add items to the cart
2. Update item quantities
3. Remove items from the cart
4. **Persist** the cart across multiple interactions or page reloads
5. **Retain** the cart state until the order is placed or the session ends



Why a shopping cart ~~cannot~~ should not be stateless

1. Persistent State:

- Must **retain** the *list of items* a user has *added* and their *respective quantities*
- **Cannot forget** the cart contents after each request (e.g., when the user navigates to a different page or refreshes the page)

1. User-Specific Data:

- Each user's cart is unique → need to **track** the *items added* by a *specific* user, either in memory or a database
- Stateless systems cannot tie specific data to a user without **re-sending** all cart data with every request, which is **inefficient**

1. Session Continuity:

- Needs to **persist** across a user's *session*, potentially *spanning* multiple requests or even days (if they come back later)
- Stateless systems cannot retain session continuity because they **discard** all information between requests

1. Performance Concerns:

- In a stateless system, the client (browser) would **need to send the entire cart data** (e.g., all items, quantities) with every request to the server → **inefficient** for larger carts and increases **network overhead**

What's the stateful version? Closure-styled Encapsulation

```
def create_calculator():
    state = 0
    # Encapsulated state

    def add(value):
        nonlocal state
        state += value
        return state

    def subtract(value):
        nonlocal state
        state -= value
        return state

    def multiply(value):
        nonlocal state
        state *= value
        return state

    def divide(value):
        nonlocal state
        if value == 0:
            return "Error: Division by zero"
        state /= value
        return state

    def reset():
        nonlocal state
        state = 0
        return state

    def get_state():
        return state

    return add, subtract, multiply, divide, reset, get_state
```

```
# Example Usage
add, subtract, multiply,
divide, reset, get_state =
create_calculator()

print(add(5))
# Output: 5
print(multiply(2))
# Output: 10
print(subtract(3))
# Output: 7
print(divide(7))
# Output: 1.0
print(get_state())
# Output: 1.0
print(reset())
# Output: 0
```

Is Closure scalable?

Scalability: Not ideal for large-scale or distributed systems.

- **Localized** and **encapsulate state** *within a single function*
- Works well for **small, isolated** systems
- Does not scale well when:
 - System needs to handle a large number of concurrent users
 - State needs to be shared or synchronized across multiple instances
- Closures also become harder to maintain and debug as the functionality grows

Best for: Small-scale, self-contained applications where each instance handles independent users or sessions.

What would be a scalable version? Immutable Data Structure

```
def add(state, value):  
    return state + value #creates and returns a new state  
  
def subtract(state, value):  
    return state - value  
  
def multiply(state, value):  
    return state * value  
  
def divide(state, value):  
    if value == 0:  
        return "Error: Division by zero", state  
    return state / value  
  
# Example Usage  
state = 0 # Initial state  
  
state = add(state, 5) # Add 5; returns a new state  
state = multiply(state, 2) # Multiply by 2  
state = subtract(state, 3) # Subtract 3  
state = divide(state, 7) # Divide by 7  
  
print(state) # Output: 1.0
```

Key Takeaways

Version	Scalability	Best Use Case
Closure-Based Stateful Calculator	Limited	Small-scale, session-based systems where each instance operates independently.
Stateful Calculator Using Generators	Limited	Small-scale systems requiring efficient, localized state handling.
Stateful Calculator with Explicit State	Moderately Scalable	Stateless systems that require functional principles and immutability.
Stateful Calculator Using Immutable Data Structure	Highly Scalable	Large-scale, distributed systems, or applications requiring horizontal scaling.

Then why OOP? Why not Immutable Data Structures?



Modeling real-world entities can make life “unreal”



Modeling a Car in FP via Pure Functions & Explicit State Passing

```
def create_car(color, engine_size):  
    return {"color": color, "engine_size": engine_size, "speed": 0}  ↗ Dictionary emulating an object  
  
def accelerate(car, increment):  
    # Create a new car speed-state with updated speed  
    new_car = car.copy()  
    new_car["speed"] += increment  
    return new_car  
  
def brake(car, decrement):  
    # Create a new car speed-state with updated speed  
    new_car = car.copy()  
    new_car["speed"] = max(0, new_car["speed"] - decrement)  
    return new_car  
  
def display_details(car):  Trivia: Same as double quote - car["color"]  
    print(f"Car Details:")  
    print(f"  Color: {car['color']}")  
    print(f"  Engine Size: {car['engine_size']}L")  
    print(f"  Speed: {car['speed']} km/h")
```

Example Usage

```
my_car = create_car("Red", 2.0)  
display_details(my_car)  
  
my_car = accelerate(my_car, 50)  
display_details(my_car)  
  
my_car = brake(my_car, 20)  
display_details(my_car)
```

Modeling Car in OOP via Encapsulation of internally shared state

```
class Car:
    def __init__(self, color, engine_size):
        self.color = color # Attribute: color of the car
        self.engine_size = engine_size # Attribute: engine size of the car
        self.speed = 0 # Attribute: current speed of the car

    def accelerate(self, increment):
        self.speed += increment
        print(f"The car accelerates by {increment} km/h. Current speed: {self.speed} km/h.")

    def brake(self, decrement):
        self.speed = max(0, self.speed - decrement)
        print(f"The car slows down by {decrement} km/h. Current speed: {self.speed} km/h.")

    def display_details(self):
        print(f"Car Details:\nColor: {self.color}\nEngine Size: {self.engine_size}L\nCurrent Speed: {self.speed} km/h")
```

Example Usage

```
def main():
    my_car = Car("Red", 2.0)
    my_car.display_details()
    my_car.accelerate(50)
    my_car.brake(20)
    my_car.display_details()
```

#optional but norm

```
if __name__ == "__main__":
    main()
```

Modeling real-world entities can be an overhead



FP in action while modeling a City via Immutable DS

```
from pyrsistent import pmap, pvector
```

```
# Immutable data structure representing the city map
```

```
city = pmap({  
    "districts": pvector([  
        pmap({"name": "District A", "population":  
        pmap({"name": "District B", "population":  
    ])  
})
```

hierarchical immutable data structure
representing a city

```
{  
  "districts": [  
    {"name": "District A", "population": 10000},  
    {"name": "District B", "population": 20000}  
  ]  
}
```

```
# Update the population of "District B"
```

```
def update_population(city, district_name, new_population):
```

```
    # Locate the district to update
```

```
    districts = city["districts"]
```

```
    updated_districts = districts.transform(  
        [i for i, distr in enumerate(districts) if  
        distr["name"] == district_name][0],  
        lambda distr: distr.set("population", new_population)  
    )
```

```
    # Update population of a district; [0]: denotes 1st match
```

```
    # Return a new city with updated districts
```

```
    return city.set("districts", updated_districts)
```

```
print("Original City:", city)  
print("Updated City:", new_city)
```

What can go wrong?

Performance Costs:

- Copying and reconstructing DS → significant **CPU and memory overhead**
- Worse if DS is:
 - a. Deeply nested
 - b. Contains a large number of elements

Garbage Collection:

- The original, unchanged data structure is no longer needed and will be garbage collected, **adding to runtime costs**

Modeling City in OOP via Encapsulation of internally shared state

```
class District:
    # Constructor to initialize a district
    def __init__(self, name, population):
        self.name = name
        self.population = population

    def update_population(self, new_population):
        self.population = new_population

    def display(self):
        return f"{self.name}: Population {self.population}"

class City:
    # Constructor to initialize a city with districts
    def __init__(self, districts):
        # Districts is a list of District objects
        self.districts = districts

    def update_district_population(self, district_name, new_population):
        # Find the district and update its population
        for district in self.districts:
            if district.name == district_name:
                district.update_population(new_population)
                break

    def display(self):
        # Display all districts
        print("City Districts:")
        for district in self.districts:
            print(district.display())
```

Modeling City in OOP via Encapsulation of internally shared state

```
# Example Usage
district_a = District("District A", 10000)
district_b = District("District B", 20000)

# Initialize the city with the districts
city = City([district_a, district_b])

# Display city information
city.display()

# Update the population of "District B"
city.update_district_population("District B", 25000)

# Display updated city information
city.display()
```



Key Takeaways

Criterion	FP with Immutable Data Structure	OOP Design
State Management	Works well for isolated or distributed state	Best for localized , complex , or interconnected state
Modeling Real-World Systems	Feels unnatural for real-world systems	Naturally represents real-world entities.
Performance	Can incur overhead from immutability	More efficient for stateful, performance-critical tasks
Scalability	Scales well for distributed systems	Scales well for object-heavy systems with local state
Collaboration	Harder to collaborate in large systems due to lack of cohesion	Easier to divide work with encapsulated classes

